RESUMEN DEL PROYECTO PARA EL DOCENTE

Banco Andino Colombia · Laboratorio PostgreSQL aumentado con IA

Bases de Datos II · Ingeniería de Sistemas y Telecomunicaciones · Universidad de Manizales

Integrantes: Juan Diego Ortiz · Adrian Daniel Argotte · Miguel Andrés Pava · Jerónimo Soto · David González

Repositorio: `lab-postgresql-banco-ia` · **Informe técnico completo:** `docs/informe_final.pdf`

1. El proyecto en pocas palabras

Construimos la **base de datos de un banco ficticio** en PostgreSQL. Tiene clientes (personas y empresas), cuentas con varios titulares, depósitos, retiros, transferencias, reversos y ajustes, todo con **contabilidad de doble partida**, usuarios internos con roles y auditoría. La llenamos con **datos 100 % sintéticos**: 10.000 clientes, 50.000 cuentas y 1.000.000 de transacciones.

Trabajamos por **9 fases con quality gate** (G0 a G8). Una fase solo se cerraba cuando había una **evidencia que se puede volver a ejecutar**. La IA la usamos para proponer; cada propuesta la revisamos nosotros, la ejecutó PostgreSQL y la validaron las pruebas. Las 46 decisiones tomadas sobre lo que propuso la IA están registradas.

Cifra clave	Valor
Fases cerradas	**9 de 9 (G0–G8) en PASS**
Modelo	28 tablas en 5 esquemas, 42 claves foráneas, 3FN
Pruebas del modelo (G3)	49/49 PASS + 4 pruebas de concurrencia
Calidad de los datos (G6)	76 pruebas · puntaje 99,6/100 · 0 fallas críticas
Rendimiento (G7)	Retiro en caja de 198,9 ms a 2,4 ms (82 veces más rápido)
Seguridad (G7)	23 pruebas de permisos por rol, 0 fallas
Reproducibilidad (G8)	Toda la base se reconstruye con 1 script y termina en PASS

2. Cómo está organizado el repositorio

Carpeta o archivo	Qué contiene
`README.md`	Cómo montar la base y estado de cada fase
`docs/`	Especificación (G1), modelo (G2), construcción (G3), datos (G4), ERD final, **informe final**, **matriz IA** y **guía de defensa**
`sql/00_…` a `sql/20_…`	Scripts numerados del modelo: esquemas, tablas, funciones, triggers, roles, pruebas
`sql/run_all.sql`	Crea todo el modelo y corre las 49 pruebas
`sql/load/`	Carga de los datos y perfil de calidad de G4
`sql/quality_gate.sql`	Las 76 pruebas de calidad de G6
`sql/perf/` y `sql/tests/`	Medición de rendimiento (G7), pruebas de permisos, concurrencia y la demo de la defensa
`sql/g8_reconstruir_todo.sql`	Reconstruye y verifica todo el laboratorio en un solo paso
`src/`	Generadores de datos sintéticos en Python (semilla fija 20260909)
`evidence/`	Resultados de cada fase (pruebas, CSV, planes EXPLAIN, antes/después)
`prompts/`	Los 12 prompts usados con la IA y sus respuestas

3. Resumen de cada fase

G0 · Entorno

Aspecto	Resumen
Objetivo	Dejar listas las herramientas y el repositorio compartido.
Qué hicimos	Instalamos PostgreSQL 18.6 y Python 3.13 en nuestro PC y configuramos el repositorio en GitHub con reglas (sin contraseñas ni archivos pesados).
Resultado	Entorno verificado con un script.
Dónde verlo	`evidence/g0_environment.txt`

G1 · Entender el negocio

Aspecto	Resumen
Objetivo	Especificar el banco antes de escribir SQL.
Qué hicimos	Definimos actores, procesos, entidades, catálogos y 54 reglas **de negocio verificables** (RN-01 a RN-54). Adoptamos 5 supuestos, por ejemplo: una cuenta bloqueada recibe dinero pero no lo saca; un titular debe tener 18 años. Identificamos 9 riesgos de consistencia.
Resultado	54 reglas (la guía pide ≥ 30), con un ERD conceptual y la matriz IA inicial.
Dónde verlo	`docs/01_especificacion.md`

G2 · Modelo lógico

Aspecto	Resumen
Objetivo	Pasar del negocio a un modelo relacional normalizado.
Qué hicimos	Lo organizamos en 5 esquemas (`ref`, `core`, `fin`, `seg`, `aud`). Revisamos la 3FN tabla por tabla y justificamos 8 desnormalizaciones controladas, como guardar el saldo en la cuenta además del ledger. La IA auditó el modelo (Prompt 2A) y encontró 12 hallazgos, que aceptamos, modificamos o rechazamos.
Resultado	28 tablas, 42 relaciones, 13 decisiones de modelado documentadas.
Dónde verlo	`docs/02_modelo_logico.md`, `docs/02_erd.png`

G3 · Construcción en PostgreSQL

Aspecto	Resumen
Objetivo	Implementar el modelo y que la base haga cumplir las reglas.
Qué hicimos	Scripts numerados 00–20. El dinero solo se mueve con **17 funciones seguras** (consignar, retirar, transferir, reversar…), que validan permisos, bloquean la cuenta y registran la doble partida en un solo paso. Creamos 32 triggers (inmutabilidad, cuadre contable, auditoría, ciclo de vida) y 5 roles de base de datos. Cada error del negocio tiene su código (BA001–BA017).
Resultado	**49/49 pruebas** (39 negativas y 10 positivas) + 4 pruebas de concurrencia con dos sesiones: doble retiro, transferencias cruzadas, doble clic y doble reverso. También en PostgreSQL 18.6.
Dónde verlo	`sql/`, `docs/03_construccion.md`, `evidence/g3_tests.txt`

G4 · Datos sintéticos

Aspecto	Resumen
Objetivo	Llenar la base con volumen realista y reproducible.
Qué hicimos	Dos generadores en Python: uno para clientes y cuentas (Faker colombiano) y otro que simula el millón de transacciones **en orden cronológico**, respetando saldo, estado y límite en cada momento. Carga masiva con `\copy`.
Resultado	10.000 clientes, 50.000 cuentas y 1.000.000 de transacciones con 2.000.000 de asientos. Perfil de **48/48 métricas**. Dos ejecuciones dan archivos idénticos (mismo SHA-256). Las distribuciones no son uniformes, como en un banco real.
Dónde verlo	`src/`, `docs/04_datos_sinteticos.md`, `evidence/g4_data_profile.csv`

G5 · Consultas SQL

Aspecto	Resumen
Objetivo	Demostrar dominio de SQL sobre los datos reales.
Qué hicimos	30 consultas (10 básicas, 10 intermedias, 10 avanzadas con funciones de ventana, CTE y percentiles), cada una con su objetivo de negocio y su explicación, más **7 validaciones cruzadas** que comprueban los resultados por dos caminos.
Resultado	30/30 funcionan sobre el millón de transacciones; 7/7 validaciones en PASS.
Dónde verlo	`sql/19_demo_queries.sql`, `evidence/g5_results.md`

G6 · Calidad de datos

Aspecto	Resumen
Objetivo	Auditar la base sin corregir nada primero y luego corregir con evidencia.

Aspecto	Resumen
Qué hicimos	76 pruebas automáticas en 12 categorías (completitud, unicidad, integridad, temporal, financiera, saldos, auditoría…). Se detectó una falla: las cargas masivas no dejaban rastro en la auditoría. Se corrigió y se repitió. Para probar que las pruebas sí detectan errores, se metieron **7 defectos** en una copia de la base.
Resultado	Antes **98,3** (1 falla ALTA) → después **99,6** (0 fallas). Los 7 defectos fueron detectados.
Dónde verlo	`sql/quality_gate.sql`, `evidence/g6_quality_before_after.md`

G7 · Rendimiento y seguridad

Aspecto	Resumen
Objetivo	Medir antes de optimizar y demostrar el mínimo privilegio.
Qué hicimos	10 consultas y 3 operaciones medidas con `EXPLAIN (ANALYZE, BUFFERS)` antes y después. Solo creamos 6 índices, cada uno justificado por una consulta; en 3 casos decidimos no crear índice. Documentamos un caso en que un índice empeoró un reporte. Para la seguridad, hicimos pruebas con un usuario de cada rol.
Resultado	Retiro: 198,9 → 2,4 ms. Inactivación de cuentas: 51,9 → 5,4 s. Costo: 86 MB. Permisos: 23/23 (8 permitidos, 15 denegados).
Dónde verlo	`evidence/g7_explain.md`, `evidence/g7_security.md`

G8 · Cierre

Aspecto	Resumen
Objetivo	Dejar el proyecto reproducible y listo para la defensa.
Qué hicimos	Un solo script reconstruye y verifica todo. El ERD final se genera leyendo la base, así que coincide con lo implementado. Escribimos el informe final, la matriz IA consolidada y la guía de defensa, con una demo en vivo de 8 pruebas.
Resultado	Reconstrucción **PASS** (3 min en PostgreSQL 16; 10 min en nuestro PC). Los datos reconstruidos son idénticos byte a byte.
Dónde verlo	`sql/g8_reconstruir_todo.sql`, `docs/erd_final.png`, `docs/informe_final.pdf`

4. Uso de la IA

Registramos 46 decisiones sobre recomendaciones de la IA: 19 aceptadas, 11 modificadas, 10 rechazadas, 5 errores de la IA corregidos y 1 aplazada. Es decir, **27 de 46 no las aceptamos tal cual.** Los tres casos más importantes:

- **Auditoría:** la IA propuso cargar los datos sin triggers, por velocidad, y no advirtió que así la carga no quedaba auditada. Lo detectó la prueba A04 de G6 y se corrigió.
- **Índices:** la IA sugirió indexar los reportes más lentos. El EXPLAIN mostró que el problema real era otro: cada retiro recorría el millón de transacciones con la cuenta bloqueada.
- **Datos:** el primer generador dejó movimientos en cuentas que se iban a inactivar y ningún rechazo por límite diario. Lo detectaron los controles de carga.

Detalle completo: `docs/matriz_ia.md` y `prompts/prompts_y_respuestas.md`.

5. Cómo revisarlo usted mismo

Desde la carpeta `lab-postgresql-banco-ia`, con PostgreSQL y Python instalados:

1. `python src/generate_transactions.py`: genera el millón de transacciones (unos 30 s; no se suben por su tamaño).
2. Crear la base `banco_andino_lab`.
3. `psql -U postgres -d banco_andino_lab -f sql/g8_reconstruir_todo.sql`: construye todo y termina en **G8 RECONSTRUCCIÓN = PASS**.
4. `psql -U postgres -d banco_andino_lab -f sql/tests/g8_demo_defensa.sql`: demo de 8 pruebas en vivo (inmutabilidad, cuadre contable, saldos, idempotencia, atomicidad, límite diario, índice y permisos). No modifica la base.